

Sistema Integrador de Ferrocarriles Españoles

Hito Final | Grupo 4

Víctor Elvira Fernández, Juan Horrillo Crespo, Sergio Velasco de Pedro

20-05-2026

Índice

1	Tren-ES	2
1.1	Supuesto elegido	2
1.2	Esquema mediador	2
1.2.1	Tabla Estacion	3
1.2.2	Tabla Municipio	3
1.2.3	Tabla Viaje	3
1.2.4	Tabla Parada	3
1.2.5	Tabla Ruta	4
1.2.6	Tabla Distancia	4
1.3	Diagrama entidad-relación del mediador	4
1.4	Formulación SQL de las consultas	4
1.5	Captura de una instancia	6
2	Esquemas Origen	10
2.1	Forma conjuntiva de las tablas origen	11
2.1.1	Municipio	13
2.1.2	Estacion	13
2.1.3	Distancia	13
2.1.4	Ruta y Parada	13
2.1.5	Viaje	14
2.1.6	wikidata_MUNICIPIO	14
2.1.7	INE_MUNICIPIO	14
2.1.8	INE_PROVINCIA	15
2.1.9	data_renfe_ESTACION	15
2.1.10	horarios_renfe_RUTA	15
2.1.11	adif_SALIDAS	15
3	Reformulación GAV y LAV de las Consultas	15
3.1	Consulta 1	15
3.1.1	Reformulación GAV	15
3.1.2	Reformulación LAV	16
3.2	Consulta 2	17
3.2.1	Reformulación GAV/LAV	17
3.3	Consulta 3	17
3.3.1	Reformulación GAV/LAV	17
4	Scraping	18
4.1	ADIF	18
4.2	RENFE	19

5	WikiData	21
5.1	Estructura de los datos obtenidos	21
5.2	Consulta SPARQL utilizada	22
5.3	Script de extracción en Python	23
5.4	Explicación del script	24
5.5	Ejecución de la consulta	24
5.6	Procesamiento de los resultados	24
5.7	Exportación a JSON	25
5.8	Exportación a CSV	25
5.9	Resultado del proceso	25
6	API	26
6.1	RENFE	26
6.1.1	La problemática: muchas estaciones	26
6.2	INE	27
7	Integración final	27
7.1	Pipelines ETL	28
7.2	Integración Híbrida	29
8	Límites y retos del sistema	29
8.1	Limitaciones del sistema	29
8.2	Retos temporales	29
8.3	Retos en la extracción de datos	30
8.4	Retos espaciales y de cruce de datos	30
8.5	Retos tecnológicos	30
8.6	Retos de integración	30
8.7	Otras limitaciones	30

1 Tren-ES

1.1 Supuesto elegido

Hemos elegido realizar la práctica sobre un supuesto propio, distinto de los propuestos. El objetivo consiste en construir un agregador que integre información acerca de la situación ferroviaria nacional. El sistema relacionará información sobre las estaciones, rutas y viajes en tren por el territorio español. El objetivo final será realizar las siguientes consultas:

1. Estaciones de tren en poblaciones de menos de 10000 habitantes.
2. Viajes entre dos estaciones a menos de 30 km entre sí.
3. Poblaciones de entre 20000 y 100000 habitantes en las que haya menos de cinco viajes programados hoy.

1.2 Esquema mediador

Para modelar la información del sistema se ha definido un esquema mediador compuesto por seis relaciones principales: **Estacion**, **Municipio**, **Viaje**, **Parada**, **Ruta** y **Distancia**. Este esquema permite representar tanto la estructura geográfica del sistema ferroviario como la planificación de trayectos y viajes.

Las relaciones del esquema mediador son las siguientes:

- Estacion(**id**, nombre, *municipio*, latitud, longitud)
- Municipio(nombre, n_habitantes, **id**, latitud, longitud, provincia, CCAA)
- Viaje(**id**, *ruta*, fecha, horario)
- Parada(*ruta*, *estacion*, n_secuencia, km_origen)
- Ruta(**id**, *origen*, *destino*, tipo)
- Distancia(*estacion1*, *estacion2*, distancia)

A continuación se describe brevemente cada una de las tablas y los atributos que la componen.

1.2.1 Tabla Estacion

La tabla **Estacion** guarda la información de las estaciones de tren registradas en el sistema. Cada estación está asociada a un municipio concreto.

Sus atributos son:

- **id**: clave primaria de la tabla. Identifica de forma única cada estación.
- **nombre**: nombre de la estación.
- **municipio**: clave foránea que referencia al atributo **id** de la tabla **Municipio**. Indica el municipio en el que se encuentra situada la estación.
- **latitud**: coordenada geográfica de latitud de la estación.
- **longitud**: coordenada geográfica de longitud de la estación.

1.2.2 Tabla Municipio

La tabla **Municipio** almacena la información básica de cada municipio incluido en el sistema. En ella se recogen tanto datos identificativos como datos demográficos y geográficos.

Sus atributos son:

- **id**: clave primaria de la tabla. Identifica de manera única cada municipio.
- **nombre**: nombre del municipio.
- **n_habitantes**: número de habitantes del municipio.
- **latitud**: coordenada geográfica de latitud del municipio.
- **longitud**: coordenada geográfica de longitud del municipio.
- **provincia**: provincia a la que pertenece el municipio.
- **CCAA**: comunidad autónoma a la que pertenece el municipio.

1.2.3 Tabla Viaje

La tabla **Viaje** almacena los viajes concretos programados en el sistema. Mientras que una ruta define un recorrido general, un viaje representa una realización específica de dicha ruta en una fecha y hora determinadas.

Sus atributos son:

- **id**: clave primaria de la tabla. Identifica de forma única cada viaje.
- **ruta**: clave foránea que referencia al atributo **id** de la tabla **Ruta**. Indica qué ruta realiza el viaje.
- **fecha**: fecha en la que está programado el viaje.
- **horario**: hora programada para la salida del viaje.

1.2.4 Tabla Parada

La tabla **Parada** almacena las estaciones por las que pasa una ruta, guardando el orden de llegada planificado en la ruta a la que pertenece. Esta tabla NO tiene en cuenta la existencia de rutas circulares (que por otra parte creemos que no existen en recorridos de alta y media distancia). Consideramos que las rutas tienen un origen y un destino, con una serie de paradas intermedias.

Sus atributos son:

- **ruta**: clave foránea que referencia al atributo **id** de la tabla **Ruta**. Forma parte de la clave primaria compuesta de la tabla.
- **estacion**: clave foránea que referencia al atributo **id** de la tabla **Estacion**. Forma parte de la clave primaria compuesta de la tabla.
- **n_secuencia**: indica el orden de la parada dentro de la ruta.
- **km_origen**: distancia en kilómetros desde el origen de la ruta hasta esa parada.

La clave primaria de **Parada** es compuesta, formada por los atributos **ruta** y **estacion**.

1.2.5 Tabla Ruta

La tabla **Ruta** representa los recorridos ferroviarios entre estaciones. Cada ruta conecta una estación de origen con una estación de destino y permite clasificar el trayecto según su tipo.

Sus atributos son:

- **id**: clave primaria de la tabla. Identifica de manera única cada ruta.
- **origen**: clave foránea que referencia al atributo **id** de la tabla **Estacion**. Indica la estación de origen de la ruta.
- **destino**: clave foránea que referencia al atributo **id** de la tabla **Estacion**. Indica la estación de destino de la ruta.
- **tipo**: tipo de ruta o servicio ferroviario asociado.

1.2.6 Tabla Distancia

La tabla **Distancia** almacena la distancia entre pares de estaciones. Esta relación se utiliza para facilitar consultas sobre proximidad geográfica entre estaciones.

Sus atributos son:

- **estacion1**: clave foránea que referencia al atributo **id** de la tabla **Estacion**. Forma parte de la clave primaria compuesta.
- **estacion2**: clave foránea que referencia al atributo **id** de la tabla **Estacion**. Forma parte de la clave primaria compuesta.
- **distancia**: distancia entre ambas estaciones.

La clave primaria de **Distancia** es compuesta, formada por **estacion1** y **estacion2**.

1.3 Diagrama entidad-relación del mediador

1.4 Formulación SQL de las consultas

Podemos formular las consultas de la siguiente manera

1. Estaciones de tren en poblaciones de menos de 10000 habitantes.

```
SELECT E.id, E.nombre AS estacion, M.nombre AS municipio
FROM Estacion AS E
INNER JOIN Municipio AS M
    ON E.municipio = M.id
WHERE M.n_habitantes < 10000;
```

2. Viajes entre dos estaciones a menos de 30 km entre sí.

Para la solución a esta consulta es importante ver a qué nos referimos con *distancia*: es la distancia euclídea directa entre las estaciones origen y destino a partir de su longitud y latitud. No tiene en cuenta el recorrido que haga, pudiendo ser este mayor que 30 km.

```
SELECT V.id
FROM Viaje AS V
INNER JOIN Ruta AS R
    ON V.ruta = R.id
INNER JOIN Distancia AS D
    ON (R.origen = D.estacion1 AND R.destino = D.estacion2) OR
    (R.origen = D.estacion2 AND R.destino = D.estacion1)
WHERE D.distancia < 30;
```

3. Poblaciones de entre 20000 y 100000 habitantes en las que haya entre uno y cinco viajes programados hoy.

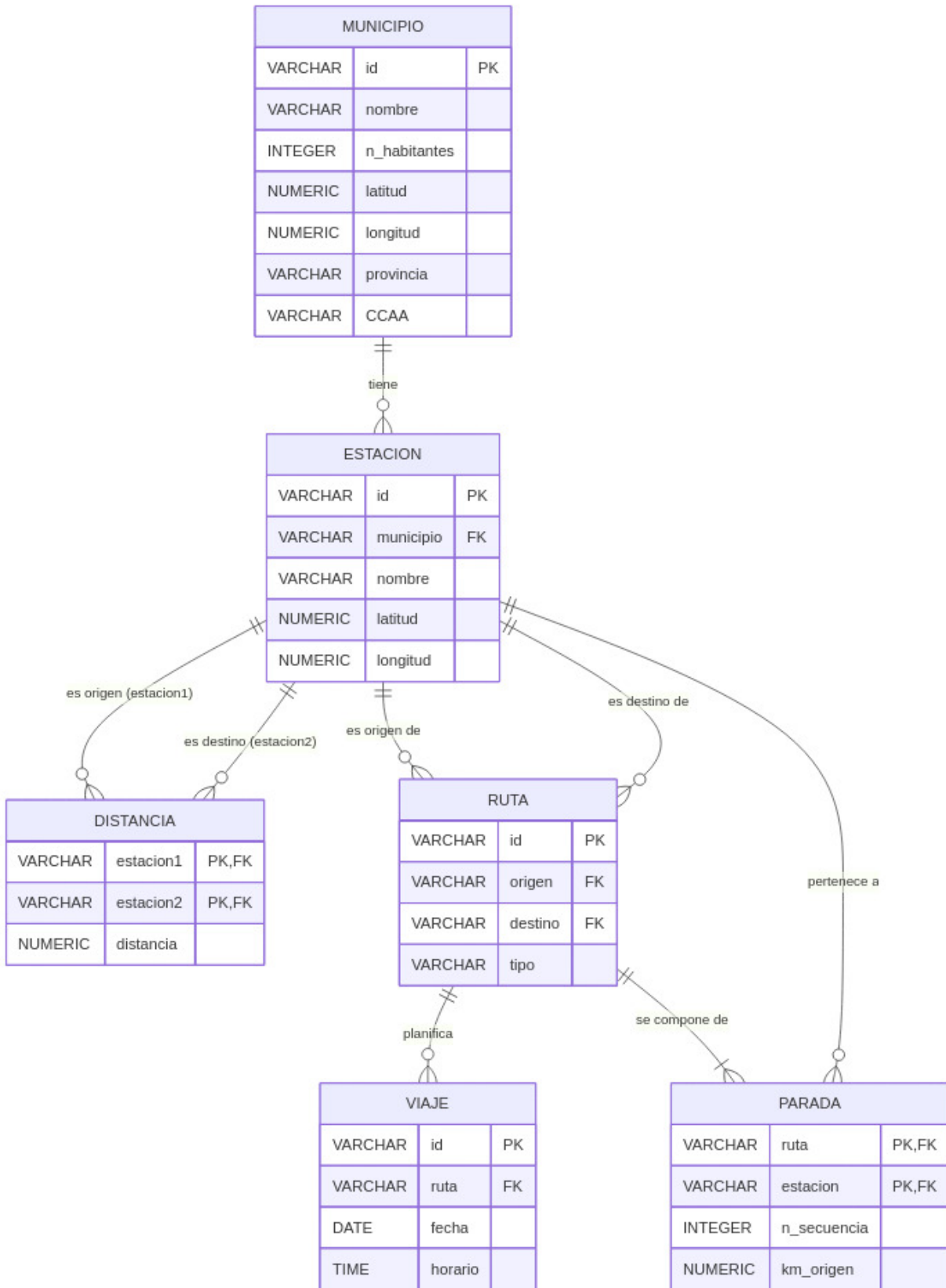


Figura 1: Esquema del mediador

Esta consulta en principio iba a ser ****Poblaciones ...** haya menos de cinco viajes programados hoy. El problema con esta consulta así formulada es que es necesario hacer un *OUTER JOIN* para obtener aquellas poblaciones en las que haya 0 viajes programados, lo que daba problemas al poner las consultas en forma conjuntiva, por lo que hemos decidido realizar esta simplificación.

```
SELECT
    M.nombre,
    M.n_habitantes,
    COUNT(P.ruta) AS total_parados
FROM
    Municipio AS M
    INNER JOIN Estacion AS E ON E.municipio = M.id
    INNER JOIN Parada AS P ON P.estacion = E.id
WHERE
    M.n_habitantes BETWEEN 20000 AND 100000
GROUP BY
    M.id,
    M.nombre,
    M.n_habitantes
HAVING
    COUNT(DISTINCT P.ruta) BETWEEN 1 AND 5;
```

1.5 Captura de una instancia

En este apartado se muestran las capturas de instancias específicas del esquema mediador:

Ruta

id	origen	destino	tipo
1	1	10	AVE
2	10	1	AVE
3	1	2	AVANT
4	2	10	AVE
5	1	3	Media Distancia
6	3	5	Media Distancia
7	6	7	Regional
8	8	10	Media Distancia
9	4	1	Media Distancia
10	9	10	Alvia
11	5	3	Media Distancia
12	7	6	Regional
13	10	8	Media Distancia
14	1	4	Media Distancia
15	10	9	Alvia

Distancia

estacion1	estacion2	distancia
1	1	0
1	2	97.07
1	3	44.45
1	4	119.1
1	5	127.2
1	6	109.56
1	7	86.34
1	8	109.66
1	9	188.05
1	10	156.8
2	1	97.07
2	2	0
2	3	128.22
2	4	165.87
2	5	224.27
2	6	132.58
2	7	154.07
2	8	58.32
2	9	165.92
2	10	59.77
3	1	44.45
3	2	128.22
3	3	0
3	4	81.52
3	5	107.58
3	6	150.94
3	7	115.02

Estacion

id	nombre	municipio_id	latitud	longitud
1	Valladolid Campo Grande	1	41.642	-4.727
2	Segovia Guiomar	2	40.91	-4.094
3	Palencia	3	42.015	-4.534
4	Burgos Rosa Manzano	4	42.367	-3.666
5	León	5	42.594	-5.582
6	Salamanca	6	40.958	-5.671
7	Zamora	7	41.515	-5.751
8	Ávila	8	40.656	-4.7
9	Soria	9	41.768	-2.468
10	Madrid Chamartín-Clara Campoamor	10	40.4722	-3.6826

Municipio

id	nombre	n_habitantes	latitud	longitud	provincia	ccaa
1	Valladolid	295639	41.6523	-4.7245	Valladolid	Castilla y León
2	Segovia	51378	40.9429	-4.1088	Segovia	Castilla y León
3	Palencia	76438	42.0095	-4.528	Palencia	Castilla y León
4	Burgos	175821	42.3439	-3.6969	Burgos	Castilla y León
5	León	121281	42.5987	-5.5671	León	Castilla y León
6	Salamanca	144436	40.9701	-5.6635	Salamanca	Castilla y León
7	Zamora	59475	41.5033	-5.7446	Zamora	Castilla y León
8	Ávila	58085	40.6566	-4.6819	Ávila	Castilla y León
9	Soria	39821	41.7633	-2.4688	Soria	Castilla y León
10	Madrid	3334730	40.4168	-3.7038	Madrid	Comunidad de Madrid

Parada

ruta	estacion	n_secuencia	km_origen
1	1	1	0
1	10	2	193
2	10	1	0
2	1	2	193
3	1	1	0
3	2	2	108
4	2	1	0
4	10	2	92
5	1	1	0
5	3	2	50
6	3	1	0
6	5	2	127
7	6	1	0
7	7	2	66
8	8	1	0
8	10	2	115
9	4	1	0
9	3	2	90
9	1	3	140
10	9	1	0
10	10	2	231
11	5	1	0
11	3	2	127
12	7	1	0
12	6	2	66
13	10	1	0
13	8	2	115
14	1	1	0
14	3	2	50
14	4	3	140
15	10	1	0
15	9	2	231

Viaje

id	ruta	fecha	horario
1	1	2026-03-24	08:30:00
2	1	2026-03-24	14:00:00
3	1	2026-03-25	18:30:00
4	2	2026-03-24	10:00:00
5	2	2026-03-25	19:00:00
6	3	2026-03-24	09:15:00
7	3	2026-03-26	17:10:00
8	4	2026-03-24	11:20:00
9	4	2026-03-25	16:40:00
10	4	2026-03-26	20:10:00
11	5	2026-03-24	07:45:00
12	5	2026-03-25	15:30:00
13	5	2026-03-26	19:20:00
14	6	2026-03-24	13:10:00
15	7	2026-03-24	18:00:00
16	7	2026-03-26	12:00:00
17	8	2026-03-24	06:50:00
18	8	2026-03-25	16:10:00
19	8	2026-03-26	21:00:00
20	9	2026-03-25	16:30:00
21	9	2026-03-26	09:40:00
22	10	2026-03-25	17:45:00
23	10	2026-03-26	07:30:00
24	11	2026-03-24	12:15:00
25	12	2026-03-26	07:55:00
26	13	2026-03-25	09:05:00
27	13	2026-03-26	13:25:00
28	14	2026-03-25	11:50:00
29	14	2026-03-26	18:05:00
30	15	2026-03-25	15:15:00

2 Esquemas Origen

En este documento se muestran los esquemas de las fuentes de datos origen de nuestro sistema integrador Tren-ES. Las fuentes de datos son las siguientes:

- Wikidata. Para obtener información de los municipios, como su nombre, código del INE, coordenadas geográficas o la población. Para extraer estos datos utilizaremos un *punto sparql*
- INE. Para conectar los códigos de los municipios con las provincias y comunidades autónomas. Para extraer estos datos utilizaremos la *API oficial*.
- Renfe. Dos distintas:
 - Renfe Data. Obtenemos información a partir de la *API de Renfe* de todas las estaciones que existen en España.
 - Renfe horarios. Un pequeño formulario desde el que se puede obtener la información de todos los horarios de tramos (Estación Origen, Estación Destino) disponibles en la web de renfe y las rutas correspondientes. Para extraer estos datos utilizaremos *web scraping*.
- ADIF. Información en tiempo real de las salidas y llegadas en cada estación. Nos permite contrastar los horarios planificados con los reales. Para extraer estos datos utilizaremos *web scraping*.

2.1 Forma conjuntiva de las tablas origen

- `wikidata_MUNICIPIO`(`codigo` INTEGER, `label` VARCHAR(255), `poblacion` INTEGER, `coordenadas` POINT) Tabla de datos extraídos de Wikidata. La tabla está formada por un campo `codigo` que contiene el identificador INE del municipio, un campo `label` con el nombre del municipio en texto legible, la columna `poblacion` que almacena el número de habitantes de ese municipio y las `coordenadas`, una tupla de tipo Point que almacena las coordenadas geográficas de dicho municipio.
- `INE_MUNICIPIO`(`cmun` INTEGER, `cpro` INTEGER, `dc` INTEGER, `nombre` VARCHAR(100)) Datos oficiales sobre municipios ofrecidos por el INE. Formados por un código de municipio `cmun`, que identifica a un municipio *dentro* de una provincia, el código de la provincia a la que pertenece, `cpro`, un dígito de control (que no nos genera interés) `dc` y el nombre en texto legible, `nombre`.
- `INE_PROVINCIA`(`cpro` INTEGER, `nombre` VARCHAR(100), `codauto` INTEGER, `ccaa` VARCHAR(100)) Datos oficiales sobre provincias ofrecidos por el INE. Es una tabla que cruza provincias con su respectiva comunidad autónoma. Formada por la información de provincia, `cpro` y `nombre`, código de la provincia y nombre en texto legible respectivamente, e información sobre la Comunidad Autónoma a la que pertenece, `codauto` y `ccaa`, código de la Comunidad Autónoma y nombre en texto legible respectivamente.
- `data_renfe_ESTACION`(`id` INTEGER, `codigo` INTEGER, `descripcion` VARCHAR(255), `latitud` NUMERIC(9,6), `longitud` NUMERIC(9,6), `direccion` VARCHAR(255), `c_p` VARCHAR(5), `poblacion` VARCHAR(100), `provincia` VARCHAR(100), `pais` VARCHAR(100)) Este esquema, se ofrece en la sección de datos abiertos de Renfe. Contiene información sobre todas las estaciones de trenes que hay en España. Se compone por, un `id` de asignación secuencial, un código de estación `codigo`, el nombre en texto legible guardado en `descripcion`, las coordenadas geográficas por separado en `latitud` y `longitud`, una dirección guardada en la columna `direccion` y el código postal asociado `c_p` y finalmente datos territoriales de la ubicación, `población`, `provincia` y `pais`.
- `horarios_renfe_HORARIO`(`recorrido` VARCHAR(255), `salida` TIME, `llegada` TIME, `duracion` INTERVAL) Datos extraídos de un formulario de la página de Renfe. El horario son viajes que van desde una estación de origen hasta una de destino. `recorrido` es la ruta que realiza ese viaje, `salida` y `llegada` son las estaciones que une dicho viaje (y que son paradas de la ruta). Duración es el tiempo en horas y minutos que tarda en completarse el viaje.
- `horarios_renfe_RUTA`(`codigo` INTEGER, `tipo` VARCHAR(20), `paradas` TEXT) Se extraen del mismo formulario, contiene información sobre una ruta en específico. Las rutas son conjuntos de estaciones que se recorren en un orden específico. Una ruta tiene un `codigo` que la identifica frente al resto, y un `tipo` de tren que la recorre (MD, AV, AVLO, ...) y `paradas` representa la lista de paradas. Sabemos que no es una representación normalizada, pero nos parecía que añadía mucha complejidad a representar los orígenes, y por motivos de simpleza lo hemos hecho así. En el esquema mediador se trata y soluciona este problema.
- `adif_SALIDAS`(`hora` TIME, `destino` VARCHAR(255), `tren` VARCHAR(100), `via` INTEGER, `estacion` VARCHAR(255)) Información accesible desde las páginas de datos en tiempo real sobre las estaciones españolas. Contiene información sobre las salidas en tiempo real. `hora` es el cuándo sale el tren de la estación, el campo `destino` contiene la estación destino de dicha salida, `tren` almacena tanto la ruta de dicho viaje que sale como el tipo de tren que la atiende, una columna `via` desde dónde es dicha salida y `estacion` la estación de la que provienen los datos.
- `adif_LLEGADAS`(`hora` TIME, `origen` VARCHAR(255), `tren` VARCHAR(100), `via` INTEGER, `estacion` VARCHAR(255)) La entidad de llegadas contiene la misma información que salidas, pero intercambiando cualquier dato por los correspondientes a las llegadas.

wikidata_MUNICIPIO		
INTEGER	CODIGO	CÓDIGO
VARCHAR	LABEL	
INTEGER	POBLACION	POBLACIÓN
POINT	COORDENADAS	

INE_MUNICIPIO		
INTEGER	CMUN	CÓDIGO DE MUNICIPIO
INTEGER	CPRO	CÓDIGO DE PROVINCIA
INTEGER	DC	DÍGITO DE CONTROL
VARCHAR	NOMBRE	

INE_PROVINCIA		
INTEGER	CPRO	CÓDIGO DE PROVINCIA
VARCHAR	NOMBRE	NOMBRE DE PROVINCIA
INTEGER	CODAUTO	CÓDIGO DE CCAA
VARCHAR	CCAA	NOMBRE COMUNIDAD AUTÓNOMA

data_renfe_ESTACION		
INTEGER	ID	
INTEGER	CODIGO	CÓDIGO
VARCHAR	DESCRIPCION	NOMBRE ESTACIÓN
NUMERIC	LATITUD	
NUMERIC	LONGITUD	
VARCHAR	DIRECCION	DIRECCIÓN
VARCHAR	C_P	C.P.
VARCHAR	POBLACION	
VARCHAR	PROVINCIA	
VARCHAR	PAIS	

horarios_renfe_HORARIO		
VARCHAR	RECORRIDO	TREN/RECORRIDO
TIME	SALIDA	
TIME	LLEGADA	
TIME	DURACION	HORAS/MINUTOS

horarios_renfe_RUTA		
INTEGER	CODIGO	CÓDIGO DE RUTA
VARCHAR	TIPO	TIPO DE TREN
TEXT	PARADAS	NOMBRE ESTACIONES

adif_SALIDAS	
TIME	HORA
VARCHAR	DESTINO
VARCHAR	TREN
INTEGER	VIA
VARCHAR	ESTACION

adif_LLEGADAS	
TIME	HORA
VARCHAR	ORIGEN
VARCHAR	TREN
INTEGER	VIA
VARCHAR	ESTACION

A partir del mediador y los esquemas de las fuentes de datos podemos acabar con los siguientes esquemas GAV:

2.1.1 Municipio

Es necesario cruzar los datos de los datos de *wikidata* y los del *INE* para poder obtener de forma correcta los códigos de provincia y comunidad autónoma.

```
Municipio(id, nombre, n_habitantes, latitud_m, longitud_m, provincia, CCAA)  $\supseteq$ 
wikidata_MUNICIPIO(id, nombre, n_habitantes, coordenadas), latitud_m=latitud(coordenadas),
longitud_m=longitud(coordenadas), INE_MUNICIPIO(cmun, cpro, v1, v2), id==cmun+cpro,
INE_PROVINCIA(cpro, provincia, v3, CCAA)
```

2.1.2 Estacion

Hay que tener en cuenta que solo debemos obtener estaciones de España.

```
Estacion(id, municipio, nombre, latitud_e, longitud_e)  $\supseteq$ 
data_renfe_ESTACION(v1, id, nombre, latitud_e, longitud_e, v2, v3, nombre_mun, v4, v5),
wikidata_MUNICIPIO(municipio, nombre_mun, v6, pais), pais==España
```

2.1.3 Distancia

En este caso solo guardamos las distancias entre las estaciones de forma unidireccional, ya que la distancia entre una estación y otra es la misma que entre la segunda y la primera, y así evitamos información repetida.

```
Distancia(estacion1, estacion2, distancia)  $\supseteq$ 
data_renfe_ESTACION(v1, estacion1, v2, latitud_1, longitud_1, v3, v4, v5, v6, pais),
data_renfe_ESTACION(v8, estacion2, v9, latitud_2, longitud_2, v10, v11, v12, v13, pais),
distancia=dist(latitud_1, latitud_2, longitud_1, longitud_2), estacion1 < estacion2,
pais==España
```

2.1.4 Ruta y Parada

En las fuentes de datos obtenemos desde *horarios_renfe_RUTA* una lista de paradas. Para poder extraer cada parada por separado debemos normalizar esta lista, por lo que nos ayudamos de una vista auxiliar. Esta

vista no se encuentra en los archivos SQL en este hito.

2.1.4.1 Vista auxiliar: Parada_aux Esta vista auxiliar no se encuentra en el esquema mediador, utilizaremos una función para normalizar la lista y poder extraer las paradas:

```
Parada_aux(codigo_ruta, num_secuencia, nombre_parada) ⊇
```

```
horarios_renfe_RUTA(lista_paradas, codigo_ruta),  
(num_secuencia, nombre_parada) =unlist(lista_paradas)
```

A partir de esta vista auxiliar podemos definir correctamente parada:

```
Parada(ruta, estacion, num_secuencia, km_origen) ⊇
```

```
Parada_aux(ruta, num_secuencia, nombre_parada),  
data_renfe_ESTACION(v1, estacion, nombre_parada, v3, v4, v5, v6, v7, v8, pais), pais==España
```

Para definir la ruta volvemos a necesitar funciones externas. Una ruta se define por un identificador propio y una tupla origen y destino de estaciones de la ruta. Para esto debemos obtener los números de secuencia mínimo y máximo de cada entrada de *Parada_aux*.

```
Ruta(id, origen, destino, tipo) ⊇
```

```
Parada_aux(id, num_secuencia, nombre_parada), nombre_origen=getMinSecuencia(num_secuencia),  
nombre_destino=getMaxSecuencia(num_secuencia), data_renfe_ESTACION(v1, origen, nombre_origen,  
v3, v4, v5, v6, v7, v8, pais), data_renfe_ESTACION(v10, destino, nombre_destino, v11, v12,  
v13, v14, v15, v16, pais), pais==España
```

2.1.5 Viaje

El horario se refiere a la hora de llegada en la estación destino del viaje específico, recogido de la web de *ADIF*.

```
Viaje(id, ruta, fecha, horario) ⊇
```

```
Parada_aux(ruta, num_secuencia, nombre_parada), nombre_origen=getMinSecuencia(num_secuencia),  
data_renfe_ESTACION(v1, estacion, nombre_origen, v3, v4, v5, v6, v7, v8, pais),  
adif_SALIDAS(fecha, horario, v10, v11, estacion), id=secuencia(), pais==España
```

No se han considerado simplificaciones para ninguna tabla. Para la tabla Municipio no se puede simplificar ya que es necesario hacer el cruce de tablas con el INE para obtener correctamente el código de provincia y Comunidad Autónoma. Para el resto de tablas no vemos simplificación posible.

A partir del esquema mediador y los esquemas de las fuentes de datos, podemos obtener los siguientes esquemas LAV:

2.1.6 wikidata_MUNICIPIO

Necesitamos concatenar las coordenadas de latitud y longitud extraídas en los esquemas GAV

```
wikidata_MUNICIPIO(codigo, label, poblacion, coordenadas) ⊆
```

```
Municipio(codigo, label, poblacion, latitud, longitud, v1, v2),  
coordenadas = coords(latitud, longitud)
```

2.1.7 INE_MUNICIPIO

Como no extraemos el dígito de control dc, no podemos obtenerlo de los esquemas del mediador.

```
INE_MUNICIPIO(cmun, cpro, dc, nombre) ⊆
```

```
Municipio(id, nombre, v1, v2, v3, v4, v5), id == cmun + cpro
```

2.1.8 INE_PROVINCIA

Como no extraemos el código de CCAA, no podemos obtenerlo de los esquemas del mediador.

INE_PROVINCIA(cpro, nombre, codauto, ccaa) $\subseteq$

Municipio(id, nombre, v1, v2, v3, v4, v5), id == cmun + cpro

2.1.9 data_renfe_ESTACION

Como estación venía de un formulario con varios campos que no extraemos, no podemos reconstruir exactamente la fuente completa a partir de nuestros esquemas.

data_renfe_ESTACION(v1, id, descripcion, latitud, longitud, v2, c_p, poblacion, provincia, pais) $\subseteq$

Estacion(id, id_mun, descripcion, latitud, longitud),
Municipio(id_mun, poblacion, v3, v4, v5, provincia, v6)

2.1.10 horarios_renfe_RUTA

Al contrario que en los esquemas GAV, debemos crear una lista a partir de nuestros esquemas.

horarios_renfe_RUTA(codigo, paradas) $\subseteq$

Parada(codigo, estacion, n_secuencia, v4), paradas = list(estacion, n_secuencia)

2.1.11 adif_SALIDAS

No podemos conseguir los campos tren y via desde nuestros esquemas, por lo que la tabla no se puede construir por completo.

adif_SALIDAS(hora, destino, tren, via, estacion) $\subseteq$

Viaje(id_viaje, ruta, fecha, hora), Ruta(ruta, origen, id_estacion_destino, tipo),
Estacion(id_estacion_destino, estacion, destino, v2, v3)

Al igual que en los esquemas GAV, no se ha considerado ninguna simplificación.

3 Reformulación GAV y LAV de las Consultas

En esta sección vamos a hacer la reformulación GAV y LAV de las consultas. Para la reformulación GAV emplearemos la descomposición de consultas y para la reformulación LAV utilizaremos el algoritmo de los *Buckets*.

3.1 Consulta 1

- Estaciones de tren en poblaciones de menos de 10000 habitantes.

La consulta se puede escribir sobre el esquema mediador como:

Q1(id, nombre_estacion, nombre_municipio):- Estacion(id, id_mun, nombre_estacion, lat_e, lon_e), Municipio(nombre_municipio, habitantes, id_mun, lat_m, lon_m, prov, ccaa), hab<10000

3.1.1 Reformulación GAV

Para hacer la reformulación GAV basta con sustituir los átomos en la consulta global por sus respectivas definiciones en función de las fuentes, a partir de la formulación GAV de las fuentes vistas en apartados anteriores.

Para el átomo *Estacion*, sustituimos por su correspondiente formulación GAV: *Estacion*(id, municipio, nombre, latitud_e, longitud_e) $\subseteq$

```
data_renfe_ESTACION(v1, id, nombre, latitud_e, longitud_e, v2, v3, nombre_mun, v4, v5),
wikidata_MUNICIPIO(municipio, nombre_mun, v6, pais), pais==España
```

E igualmente con *Municipio*, que queda: *Municipio*(id, nombre, n_habitantes, latitud_m, longitud_m, provincia, CCAA) $\subseteq$

```
wikidata_MUNICIPIO(id, nombre, n_habitantes, coordenadas), latitud_m=latitud(coordenadas),
longitud_m=longitud(coordenadas), INE_MUNICIPIO(cmun, cpro, v1, v2), id==cmun+cpro,
INE_PROVINCIA(cpro, provincia, v3, CCAA)
```

Incluimos dichas formulaciones en la consulta global, añadiendo la restricción de habitantes, que da como resultado:

```
Q1'(id, nombre_estacion, nombre_municipio):- data_renfe_ESTACION(v1, id, nombre_estacion,
lat_e, lon_e, v2, v3, nom_mun, v4, v5), wikidata_MUNICIPIO(id_mun, nombre_municipio, v6, v7),
wikidata_MUNICIPIO(id_mun, nombre_municipio, hab, coord), lat_m=latitud(coord),
lon_m=longitud(coord), INE_MUNICIPIO(cmun, cpro, v8, v9),
INE_PROVINCIA(cpro, prov, v10, ccaa),
id_mun=cmun+cpro, hab<10000
```

Por último, podemos ver que el átomo *wikidata_MUNICIPIO* se encuentra repetido, por lo que podríamos pensar en simplificar para eliminar redundancias. Podríamos hacer mappings de contención para ver equivalencias, pero es evidente que, dado que v6 y v7 no se utilizan, podemos eliminar el primer átomo, quedando la consulta final como:

```
Q1'(id, nombre_estacion, nombre_municipio):- data_renfe_ESTACION(v1, id, nombre_estacion,
lat_e, lon_e, v2, v3, nom_mun, v4, pais), wikidata_MUNICIPIO(id_mun, nombre_municipio,
hab, coord), lat_m=latitud(coord), lon_m=longitud(coord),
INE_MUNICIPIO(cmun, cpro, v8, v9), INE_PROVINCIA(cpro, prov, v10, ccaa),
id_mun=cmun+cpro, hab<10000, pais=España
```

3.1.2 Reformulación LAV

Para hacer la reformulación LAV debemos utilizar el algoritmo basado en buckets.

1. Creamos y llenamos los buckets: como hay dos átomos en la consulta, creamos dos buckets. Para que una vista forme parte de un bucket es necesario que cumpla tres condiciones:
 - a. Un átomo de la vista y el átomo g para el cual construimos el bucket afectan a la misma relación.
 - b. Los predicados (condiciones) de la vista y los de la consulta Q son mutuamente satisfacibles.
 - c. Si en g aparece alguna variable cabecera de la consulta Q, entonces también debe aparecer en la cabecera de la vista.

Siguiendo las normas de construcción de buckets, para el primer átomo, su bucket estará compuesto únicamente por *wikidata_ESTACION*, que se añade al bucket. El segundo bucket estará compuesto únicamente por la fuente *wikidata_MUNICIPIO*, que se añade al segundo bucket.

2. Reformulación como producto cartesiano: hacemos el producto cartesiano entre los buckets. Como solo hay una posibilidad por bucket solo podremos hacer una reformulación. Si nos fijamos, esta reformulación será la misma que la reformulación GAV:

```
Q1''(id, nombre_estacion, nombre_municipio):- data_renfe_ESTACION(v1, id, nombre_estacion,
lat_e, lon_e, v2, v3, nom_mun, v4, v5), wikidata_MUNICIPIO(id_mun, nombre_municipio, v6, v7),
wikidata_MUNICIPIO(id_mun, nombre_municipio, hab, coord), lat_m=latitud(coord),
lon_m=longitud(coord), INE_MUNICIPIO(cmun, cpro, v8, v9),
INE_PROVINCIA(cpro, prov, v10, ccaa), id_mun=cmun+cpro, hab<10000
```


3. Posible simplificación: como ya vimos anteriormente, es posible simplificar la consulta, por lo que el resultado final es:

```
Q1'(id, nombre_estacion, nombre_municipio):- data_renfe_ESTACION(v1, id, nombre_estacion,
lat_e, lon_e, v2, v3, nom_mun, v4, v5), wikidata_MUNICIPIO(id_mun, nombre_municipio, hab,
coord), lat_m=latitud(coord), lon_m=longitud(coord), INE_MUNICIPIO(cmun, cpro, v8, v9),
INE_PROVINCIA(cpro, prov, v10, ccaa), id_mun=cmun+cpro, hab<10000
```

Como ya hemos visto, la reformulación LAV y GAV coinciden aunque hayamos utilizado diferentes algoritmos para su construcción. Es por esto que para las siguientes dos consultas únicamente se dejará indicada la reformulación final.

3.2 Consulta 2

Esta consulta tiene un OR para poder trabajar con la tabla distancia. Es por ello que la consulta se define como:

$$Q2 = Q2_a \cup Q2_b$$

Estas subconsultas se pueden escribir sobre el esquema mediador como:

```
Q2a(id, ruta, infoViaje1, infoViaje2, municipio_origen):- Viaje(id, ruta, infoViaje1, infoViaje2),
Ruta(ruta, origen, destino, v3), Distancia(origen, destino, dist),
Estacion(origen, id_mun, v4, v5, v6), Municipio(municipio_origen, v7, id_mun, v8, v9, v10, v11),
dist<30
```

```
Q2b(id, ruta, infoViaje1, infoViaje2, municipio_origen):- Viaje(id, ruta, infoViaje1, infoViaje2),
Ruta(ruta, origen, destino, v3), Distancia(destino, origen, dist),
Estacion(destino, id_mun, v4, v5, v6), Municipio(municipio_origen, v7, id_mun, v8, v9, v10, v11),
dist<30
```

3.2.1 Reformulación GAV/LAV

No hemos sido capaces de hacer ninguna de las formulaciones correctamente por su extensión

3.3 Consulta 3

Esta consulta se puede escribir en el esquema mediador como:

```
Q3(nombre, numeroParadas, numeroHabitantes):-
Municipio(id, nombre, n_habitantes, v1, v2, v3, v4),
Estacion(idE, id, v5, v6, v7),
Parada(ruta, idE, v8, v9),
Viaje(v10, ruta, CURRENT_DATE, v11),
n_habitantes >= 20000,
n_habitantes <= 100000,
COUNT(DISTINCT ruta) >= 1,
COUNT(DISTINCT ruta) <= 5.
```

3.3.1 Reformulación GAV/LAV

No hemos sido capaces de hacer ninguna de las formulaciones correctamente por su extensión

4 Scraping

4.1 ADIF

Desde la página web de consulta de horarios de estaciones en tiempo real de [ADIF](#) hemos extraído mediante scraping las tablas de las fuentes `adif_SALIDAS` y `adif_LLEGADAS`. No ha sido sencillo, ya que se trata de una aplicación web dinámica que dificulta el acceso automatizado a los datos, lo cual ha supuesto un reto para la construcción del wrapper. La implementación del scraping se puede ver en el archivo `'scraping_adif.py'`. Este fichero actúa a modo demo, por lo que no hay ningún tipo de interacción con el usuario. Simplemente accede a una página (la estación de Valladolid) y extrae las dos tablas. A continuación se procede a explicar la lógica y detalles técnicos:

Una petición HTTP básica no era suficiente para acceder al contenido real, ya que la aplicación respondía con un documento HTML indicando que el acceso no estaba permitido:

```
<HTML>
<HEAD>
<TITLE>Access Denied</TITLE>
</HEAD>
<BODY>
<H1>Access Denied</H1>
```

```
You don't have permission to access
</HTML>
```

por lo que tuvimos que rechazar una primera aproximación usando los módulos `requests` con `BeautifulSoup`. Debido a la naturaleza de la fuente de ADIF, necesitamos interactuar con una interfaz web dinámica. Por ello, probamos a utilizar el `scraper` como un `wrapper` manual que accede a la fuente, materializa el DOM y extrae los datos estructurados para las tablas `adif_SALIDAS` y `adif_LLEGADAS`. Por tanto, probamos a usar el módulo `playwright`, desarrollado por Microsoft y mucho más potente.

El módulo proporciona una interfaz muy sencilla para manejar versiones “headless” de un navegador, con el objetivo de simular la navegación de un usuario real, pudiendo interactuar con elementos de la aplicación sin simplemente limitarse a la respuesta de una petición básica. Aún así, el sistema de detección de “bots” de la web seguía respondiendo con ficheros html de rechazo, pues parece ser que también era capaz de identificar que se trataba de un navegador “headless”. Pudimos solucionar este problema creando un nuevo “contexto” de navegador que simulase una ventana gráfica, pero sin que de verdad esta existiese; gracias a esto hemos sido capaces de acceder a la información.

El flujo del programa es muy simple:

1. Se crea una “ventana” virtual y una página sobre esta, posteriormente se navega a la ruta (para el caso de valladolid, <https://www.adif.es/w/10600-valladolid-c.-g.>) y se espera a que todo el DOM esté en estado *attached* para asegurar que los elementos a los que queremos acceder están disponibles en el momento de buscarlos, debido a la estructura de HTML de los datos de la fuente.
2. Dependiendo de si se quieren extraer las tablas de llegadas o salidas, hay que pulsar un botón de “salidas” o no (la opción de “llegadas” parece que viene seleccionada por defecto). Este es el único paso diferente, a partir de aquí el programa es igual para ambos salvo alguna pequeña variante de notación. Para acceder a la tabla hay que filtrar por el identificador `#horas-trenes-estacion-(salidas|llegadas)` dependiendo de cual sea la fuente que se quiera extraer.
3. La extracción de información es la misma para ambas tablas, pues se utilizan los mismos identificadores para las columnas. Un caso particular es el de la estación de origen o destino, ya que en el HTML aparece con el mismo identificador `col-destino` independientemente de si se está consultando la tabla de salidas o la de llegadas. Esto supone una pequeña heterogeneidad semántica, ya que el mismo identificador técnico de la fuente puede representar información distinta según el contexto. Por ello, el programa interpreta este campo dependiendo de la tabla que se esté extrayendo.

Filtrar por:

● AV / Media distancia / Larga distancia

CONSULTAR

Salidas Llegadas

HORA DE SALIDA	DESTINO	TREN	VÍA
23:19 23:32	Palencia	> RF - MD 18007	4
① Compartido: Proximidad 38007 Medina del Campo - Palencia			
06:23	Madrid Chamartín Clara Campoamor	> RF - AVANT 08058	
06:26	Madrid Príncipe Pío	> RF - MD 18066	4
① Compartido: Proximidad 38066 Palencia - Medina del Campo			

RF - AVANT

Figura 2: Captura de pantalla de la página de ADIF

Al ser una tabla, extraer los datos consiste en iterar sobre todos los objetos de tipo `tr` y acceder a cada elemento `td` por columna. Sin embargo, no todas las filas representan viajes. Hay dos tipos de filas: las que contienen la información que nos interesa y las filas amarillas, que proporcionan información sobre problemas en la infraestructura. Estas últimas no tienen el mismo formato ni representan tuplas válidas para las fuentes `adif_SALIDAS` y `adif_LLEGADAS`, por lo que deben tratarse de forma separada para evitar errores y no introducir ruido en los datos extraídos.

- Finalmente, se serializan los datos extraídos en un CSV usando la función de la librería estándar `csv.DictWriter`. Esta serialización no es únicamente una operación de guardado, sino que materializa el resultado del wrapper: las filas HTML extraídas se transforman en registros estructurados acordes al esquema de las fuentes origen `adif_SALIDAS` y `adif_LLEGADAS`.
- En resumen, el scraping de ADIF puede verse como la construcción de un wrapper manual sobre una fuente web semiestructurada. El programa se encarga de acceder a la página, interpretar su estructura HTML, resolver pequeñas heterogeneidades de la fuente y limpiar aquellas filas que no se ajustan al esquema esperado, generando finalmente datos estructurados que pueden ser utilizados por el sistema integrador.

4.2 RENFE

Hemos utilizado scraping sobre la web de [renfe horarios](#). Para acceder a la información ha sido necesario construir un wrapper que interactúe con el formulario de RENFE, ya que la fuente no expone directamente los datos como una tabla descargable ni como una API sencilla. En este caso, la fuente impone un patrón de acceso basado en los campos origen, destino y fecha, por lo que se ha utilizado `playwright` para simular la interacción con el formulario y `BeautifulSoup` para procesar el HTML resultante.

El scraping realizado se puede ver en el archivo `scraping_renfe_horarios.py`, y el flujo del programa es el siguiente:

- Se introduce al programa la información relativa al formulario con el siguiente formato (que impone la web): `[cod_estacion_origen]-[cod_estacion_destino]-[dia]-[mes]-[año]`, de tal forma que para obtener los recorridos entre Valladolid Campo Grande (estación número 10600) y Palencia (estación número 14100)

```
{bash} $ echo "10600-14100-20-04-2026" | python3 scraping_renfe_horarios.py
```

2. Se accede a la web y, dentro de ella, al *iframe* que contiene el formulario. Si no se accede a este *iframe*, no es posible obtener los elementos HTML necesarios para realizar la consulta. Este paso forma parte del programa de extracción del *wrapper*, ya que es necesario localizar dentro del DOM el formulario que permite consultar la fuente. Una vez localizado, se rellenan los datos de acuerdo con la entrada del programa y se hace clic en el botón **BUSCAR**.
3. El *iframe* cambia y muestra una tabla como la de la imagen de debajo. Accedemos a cada línea de la tabla y obtenemos el primer valor, que redirige a otra página web mediante una función JavaScript y un hipervínculo.

Origen: VALLADOLID-CAMPO GRANDE

Destino: PALENCIA

Trenes para el día: Lunes 20 Abril 2026

← Día Antes

Día Después →

Trayecto Inverso

Tren / Recorrido	Salida	Llegada	Duración	Precio desde*	Prestaciones	Accesible
> 12101 REGIONAL	07.00	07.45	45 min.	> Consultar y comprar	> Ver	> Ver
> 04073 ALVIA	08.01	08.34	33 min.	> Consultar y comprar	> Ver	> Ver
> 04261 ALVIA	08.07	08.32	25 min.	> Consultar y comprar	> Ver	> Ver
> 38401 PROXIMIDAD	09.52	10.38	46 min.	> Consultar y comprar	> Ver	> Ver
> 18101 REG.EXP. >	09.52	10.52	1 h.	> Consultar y comprar	> Ver	> Ver
> 04091 ALVIA	10.18	10.43	25 min.	> Consultar y comprar	> Ver	> Ver
> 05071 AVE	10.56	11.19	23 min.	> Consultar y comprar	> Ver	> Ver
> 38061 PROXIMIDAD >	11.50	12.46	56 min.	> Consultar y comprar	> Ver	> Ver
> 18061 MD >	11.50	12.46	56 min.	> Consultar y comprar	> Ver	> Ver
> 18211 PROXIMIDAD	12.25	13.13	48 min.	> Consultar y comprar	> Ver	> Ver

4. Normalizamos el hiperenlace para que el valor extraído del HTML sea compatible con el formato esperado por la fuente. Se trata de resolver una heterogeneidad sintáctica producida por saltos de línea y espacios en blanco en la representación de la URL. Para ello, se eliminan los saltos de línea y se sustituyen los espacios en blanco por `%20`, de tal forma que la url:

20

cerrar ventana ✕

Recorrido › › Paradas del Tren

ALVIA (N°04073)

ESTACIÓN
MADRID-CHAMARTÍN-CLARA CAMPOAMOR
VALLADOLID-CAMPO GRANDE
PALENCIA
AGUILAR DE CAMPOO
REINOSA
TORRELAVEGA
SANTANDER

Duración del trayecto entre y **VALLADOLID-CAMPO GRANDE** y **PALENCIA**: 33 min.

Equivale a la imagen de debajo.

5. Accedemos a estas URL (una por ruta) y obtenemos el listado de paradas.
6. Accedemos a dos archivos CSV, correspondientes a los esquemas origen de *horarios_renfe_RUTA* y *horarios_renfe_HORARIO*, y rellenamos los datos extraídos. Esta serialización permite transformar la información obtenida desde las páginas HTML en registros estructurados compatibles con el sistema integrador. Accedemos a estos archivos en modo *append*: si no existen se crean y, si existen, se añade la nueva información al final. De esta forma, podemos realizar muchas llamadas al programa para diferentes rutas sin sobrescribir los datos anteriores.

Por último, se ha añadido la opción *-verbose* para obtener logs del programa y facilitar la revisión del proceso de extracción.

5 WikiData

Los datos de los municipios de España los vamos a obtener de **WikiData**, mediante un script de Python que utiliza el servicio de consultas SPARQL proporcionado por la propia plataforma.

Esta fuente se ha escogido porque WikiData ofrece información estructurada, accesible y fácilmente consultable sobre los municipios de España.

En concreto, necesitamos recuperar el nombre del municipio, su código oficial, la población y sus coordenadas geográficas.

5.1 Estructura de los datos obtenidos

La información que queremos obtener desde WikiData tiene la siguiente estructura:

Columna	Tipo
CODIGO	text
LABEL	text
POBLACION	numeric
COORDENADAS	Point

El campo **CODIGO** representa el código oficial del municipio. El campo **LABEL** contiene el nombre del municipio en español. El campo **POBLACION** almacena el número de habitantes y el campo **COORDENADAS** contiene la localización geográfica del municipio en formato **Point**.

5.2 Consulta SPARQL utilizada

Para realizar la extracción de datos se ha utilizado la siguiente consulta SPARQL:

```
SELECT ?codigo ?label ?poblacion ?coordenadas WHERE {
  ?municipio (wdt:P31/(wdt:P279*)) wd:Q2074737;
    wdt:P772 ?codigo;
    wdt:P1082 ?poblacion;
    wdt:P625 ?coordenadas.

  SERVICE wikibase:label {
    bd:serviceParam wikibase:language "es".
    ?municipio rdfs:label ?label.
  }
}
```

ORDER BY ?codigo

La consulta selecciona aquellas entidades que son municipios de España o pertenecen a alguna subclase relacionada. Para cada municipio se recuperan cuatro atributos principales: el código oficial, el nombre, la población y las coordenadas.

La expresión:

```
?municipio (wdt:P31/(wdt:P279*)) wd:Q2074737;
```

permite seleccionar municipios teniendo en cuenta tanto las instancias directas como aquellas entidades clasificadas mediante subclases. Esto es necesario porque en *WikiData* no están clasificados los municipios de forma uniforme.

A continuación, se extraen las propiedades necesarias:

```
wdt:P772 ?codigo;
wdt:P1082 ?poblacion;
wdt:P625 ?coordenadas.
```

Estas propiedades corresponden al código oficial del municipio, la población y las coordenadas geográficas.

Además, se utiliza el servicio de etiquetas de WikiData:

```
SERVICE wikibase:label {
  bd:serviceParam wikibase:language "es".
  ?municipio rdfs:label ?label.
}
```

Gracias a esto, se obtiene el nombre del municipio en español, evitando trabajar directamente con identificadores internos de WikiData.

5.3 Script de extracción en Python

Para automatizar el proceso se ha desarrollado el siguiente script en Python:

```
import requests
import csv
import json

WIKIDATA_URL = "https://query.wikidata.org/sparql"

# Consulta para WikiData
query = """
SELECT ?codigo ?label ?poblacion ?coordenadas WHERE {
    ?municipio (wdt:P31/(wdt:P279*)) wd:Q2074737;
        wdt:P772 ?codigo;
        wdt:P1082 ?poblacion;
        wdt:P625 ?coordenadas.

    SERVICE wikibase:label {
        bd:serviceParam wikibase:language "es".
        ?municipio rdfs:label ?label.
    }
}
ORDER BY ?codigo
"""

# Importante: sin el User-Agent, Wikidata puede bloquear la consulta
headers = {
    "Accept": "application/sparql-results+json",
    "User-Agent": "Tren-ES/0.1"
}

# Realiza la consulta
response = requests.get(
    WIKIDATA_URL,
    params={"query": query, "format": "json"},
    headers=headers,
    timeout=60
)

data = response.json()

rows = []

# Procesar los resultados
for item in data["results"]["bindings"]:
    rows.append({
        "codigo": item["codigo"]["value"],
        "label": item["label"]["value"],
        "poblacion": int(float(item["poblacion"]["value"])),
        "coordenadas": item["coordenadas"]["value"],
    })
```

```

# Guardar como JSON
with open("municipios_wikidata.json", "w", encoding="utf-8") as f:
    json.dump(rows, f, ensure_ascii=False, indent=2)

# Guardar como CSV para SQL
with open("municipios_wikidata.csv", "w", encoding="utf-8", newline="") as f:
    writer = csv.DictWriter(
        f,
        fieldnames=["codigo", "label", "poblacion", "coordenadas"]
    )
    writer.writeheader()
    writer.writerows(rows)

```

5.4 Explicación del script

En primer lugar, se importan las librerías necesarias y se define `WIKIDATA_URL` para poder hacer la conexión. `##` Cabeceras de la petición

Una parte importante del script es la definición del `User-Agent`, puesto que, sin ella, *WikiData* bloquea la consulta. La parte del `Accept` determina el formato que se quiere recibir, en este caso, JSON.

```

headers = {
    "Accept": "application/sparql-results+json",
    "User-Agent": "Tren-ES/0.1"
}

```

5.5 Ejecución de la consulta

La consulta se ejecuta mediante una petición `GET`, con un `timeout` de 60 segundos.

```

response = requests.get(
    WIKIDATA_URL,
    params={"query": query, "format": "json"},
    headers=headers,
    timeout=60
)

```

Después, la respuesta se convierte a JSON:

```
data = response.json()
```

De esta forma, los datos devueltos por *WikiData* pueden procesarse como estructuras propias de Python.

5.6 Procesamiento de los resultados

Los resultados de *WikiData* se encuentran dentro de la estructura `results.bindings`. Cada elemento representa un municipio y contiene los valores solicitados en la consulta.

Para transformar esta estructura en una lista más sencilla, se recorre cada resultado y se crea un diccionario con los campos necesarios:

```

rows = []

for item in data["results"]["bindings"]:

```



```
rows.append({
    "codigo": item["codigo"]["value"],
    "label": item["label"]["value"],
    "poblacion": int(float(item["poblacion"]["value"])),
    "coordenadas": item["coordenadas"]["value"],
})
```

En este paso se extraen únicamente los valores reales de cada campo, accediendo a la clave `value`.

La población se convierte a número entero mediante:

```
int(float(item["poblacion"]["value"]))
```

Es importante transformar los datos a `int` para asegurarse de que son números y no textos.

5.7 Exportación a JSON

Una vez procesados los resultados, se guardan en un archivo JSON. Con `ensure_ascii=False` mantenemos las ñ. Con `indent=2` se facilita el debug al meter indentaciones:

```
with open("municipios_wikidata.json", "w", encoding="utf-8") as f:
    json.dump(rows, f, ensure_ascii=False, indent=2)
```

5.8 Exportación a CSV

Además del archivo JSON, el script genera un archivo CSV, que es el que se utilizará en una base de datos relacional. Con `fieldnames` se definen los nombres de las columnas, con `writeheader()` se escribe la cabecera y con `writerows(rows)` las filas:

```
with open("municipios_wikidata.csv", "w", encoding="utf-8", newline="") as f:
    writer = csv.DictWriter(
        f,
        fieldnames=["codigo", "label", "poblacion", "coordenadas"]
    )
    writer.writeheader()
    writer.writerows(rows)
```

5.9 Resultado del proceso

Tras ejecutar el script, se obtienen dos archivos:

```
municipios_wikidata.json
municipios_wikidata.csv
```

Ambos archivos contienen la misma información, pero están pensados para usos diferentes. El archivo JSON resulta útil para conservar los datos en un formato más legible y se ha utilizado para hacer comprobaciones básicas a mano. El archivo CSV, en cambio, está orientado a la carga de datos en SQL y a su integración con el resto de fuentes del sistema.

6 API

6.1 RENFE

Para obtener la información de las estaciones necesitamos hacer uso de la API oficial de [data RENFE](#), más concretamente la asociada al dataset con el [listado completo de estaciones](#). Para hacer esta búsqueda en primer lugar decidimos no incluir todas aquellas estaciones que fuesen de cercanías o feve, pero nos encontramos con un problema: al no incluir estaciones de cercanías estábamos excluyendo todas aquellas estaciones que tuvieran el servicio, incluyendo las dos grandes estaciones de Madrid: Chamartín y Puerta de Atocha, por lo que finalmente decidimos excluir únicamente aquellas estaciones que tuvieran servicio feve.

El archivo en su origen tenía los siguientes campos:

Columna	Tipo
CODIGO	numeric
DESCRIPCION	text
LATITUD	numeric
LONGITUD	numeric
DIRECION	text
CP	numeric
POBLACION	text
PROVINCIA	text
PAIS	text
CERCANIAS	text
FEVE	text
COMUN	text

Para hacer la consulta a la api decidimos utilizar la consulta via sentencia sql para mayor control, ya que la documentación de la api era bastante poco clara (los ejemplos estaban hechos en Python2). La consulta que acabamos realizando es la siguiente:

```
SELECT "_id" as "ID", "CODIGO", "DESCRIPCION", "LATITUD", "LONGITUD", "DIRECION"
AS "DIRECCION", "CP", "POBLACION", "PROVINCIA", "PAIS"
FROM "783e0626-6fa8-4ac7-a880-fa53144654ff"
WHERE "FEVE" = 'NO'
```

Con lo que acabamos con los campos vistos en los esquemas de origen, listo para su utilización por el esquema mediador. Los campos que no hemos escogido no los hemos visto necesarios para el desarrollo de la práctica.

Hay que tener en cuenta que ciertas estaciones por algún motivo carecen de algunos campos como el código postal o la dirección. Tendremos que realizar una limpieza de estas estaciones al realizar la integración. Al final, obtenemos un listado con 870 estaciones.

6.1.1 La problemática: muchas estaciones

Tras cargar el listado de estaciones nos encontramos con un problema: encontrar las rutas necesarias entre dos estaciones via scraping. El acto de realizar este scraping entre dos estaciones tarda entre 3 y 20 segundos dependiendo del número de rutas. Puede parecer poco tiempo pero haciendo cálculos, necesitamos combinar las 870 estaciones de dos en dos:

$$\binom{870}{2} = \frac{870!}{2!868!} = \frac{870 * 869}{2} = 378015 \text{ combinaciones}$$

Con lo que obtenemos un rango de entre 13 y 85 días aproximadamente solo obteniendo los datos de las rutas para un día concreto. Esto es inviable, y es un problema que deberemos solucionar a la hora de integrar la información y realizar las consultas propuestas.

6.2 INE

El Instituto Nacional de Estadística (INE) proporciona datos de todo tipo. La información sobre economía, geografía o demografía se encuentra entre lo más importante. Recientemente el INE comenzó a ofrecer una alternativa a su portal de datos para descargar información, la API [Tempus](#). Desde esta API, entre otras cosas, se puede acceder al catálogo de Variables, Tablas, Series o Publicaciones, y al contenido de estos.

Nuestro objetivo es acceder a las variables del INE que cruzan los nombres de municipio con su código y el de la provincia en el que se encuentran, así mismo también nos interesa guardar la Comunidad Autónoma (CCAA) a la que pertenece dicha provincia, por lo que buscamos las variables Municipio y Provincia.

Para acceder a los valores de una variable es necesario conocer su código. Este catálogo de variables se encuentra en el endpoint **VARIABLES**. Las variables Municipio y Provincia corresponden a los códigos 19 y 115 respectivamente.

Como curiosidad, el contenido de **VARIABLES** no son datos en si, sino los nombres que se usan para dividir y cruzar datos de las series o tablas; por ejemplo, mostrar la evolución del PIB por provincias, o el número de personas paradas en cada año, por sexo, por nivel de formación digital...

Los valores se obtienen con una simple request http sobre el endpoint **VALORES_VARIABLE/{CODIGO}**. Para obtener la información sobre la CCAA de cada provincia es necesario solicitar información más completa. Para ello, se ha de añadir el parámetro **det** (detalle) con valor 2 (el máximo).

Al realizar el script nos encontramos con que la API no se comportaba como se esperaba. Según la documentación, existe un parámetro de página para que los datos se manden de 500 en 500. Sin embargo, por mucho probar nunca funcionó. La API es muy lenta, parece muy mal optimizada, y al descargar los aproximadamente 8000 municipios de España se demoraba varios minutos. Al final, no pudimos resolverlo, esto hizo que tomásemos la decisión de almacenar estos datos como un Warehouse, ya que no tendría sentido trabajar con semejante latencia para cada consulta que involucrase a los municipios.

7 Integración final

Hemos optado por un sistema integrador híbrido, combinando el Warhousing con técnicas de Integración Virtual. La motivación principal de esta hibridación es principalmente la diferencia entre las frecuencias de actualización de los datos. Por un lado tenemos fuentes como los datos de municipios y provincias proporcionados por el INE, que se actualizan cada mucho tiempo, frente a otras como las tablas de salidas de trenes en tiempo real en la página web de Adif, que se actualizan en periodos de segundos.

El *tech stack* está formado por:

- Apache Airflow. Para extractores y pipelines.
- MariaDB. Base de datos relacional que almacena el Warehouse.
- GraphQL. Resolución de consultas sobre el esquema mediador.

A continuación se muestra una representación gráfica del sistema

Tren-ES

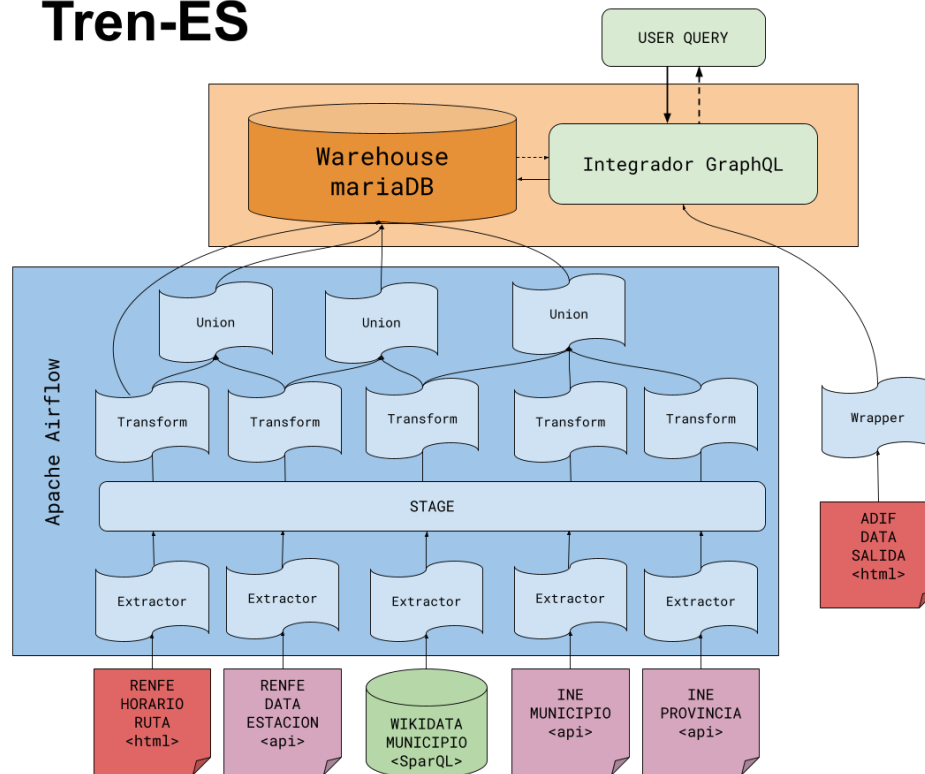
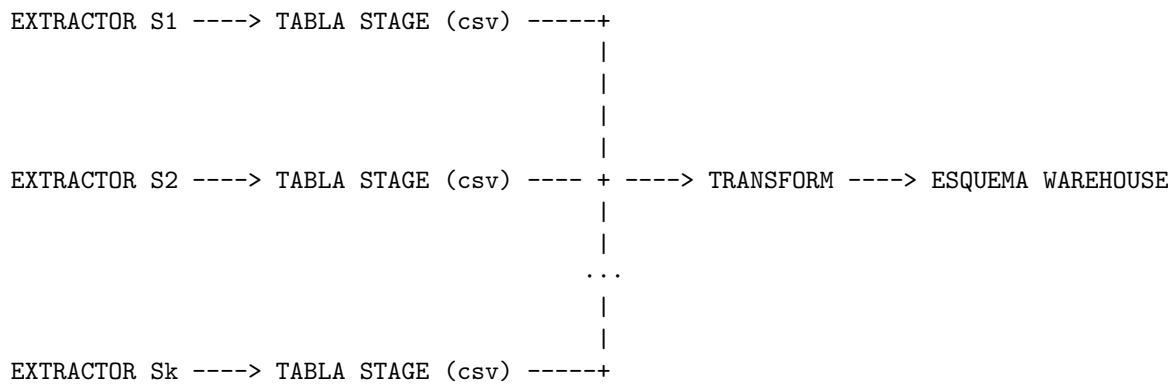


Figura 4: Diagrama sistema integrador

7.1 Pipelines ETL

Los pipelines de extracción, transformación y carga de los datos se han implementado como DAGs de Airflow. El flujo es el siguiente:



Cada fuente de datos tiene un DAG para la extracción y transformación a los esquemas definidos en los

Esquemas Origen. Estos DAG producen unas tablas intermedias llamadas tablas Stage que nos permiten resolver los Esquemas del Mediador del Warehouse sin tener que repetir dichas extracciones. Por tanto el pipeline completo consiste en **Extracción > Stage > Transformación > Carga**.

Los DAG de extracción están programados para ejecutarse de forma automática según las necesidades esperadas (diariamente, mensual, anual), además, se nos ocurrió implementar una serie de disparadores que lanzasen los DAG de transformación y carga del Warehouse una vez terminase la extracción de todas las fuentes origen de las que dependa cada esquema, pero no tuvimos tiempo de pensarlo e implementarlo bien.

7.2 Integración Híbrida

Dependiendo de la consulta del usuario, si el sistema detecta que puede usar la fuente de datos en tiempo real de la web de Adif, este tratará de obtener la información mediante un scrapping web desdoblando la consulta por Integración Virtual. En ocasiones no es posible utilizar esa fuente (acceso denegado, petición errónea, etc), en esos casos, el sistema usa una fuente de datos menos actualizada/fiable almacenada en el Warehouse para dar respuesta.

Hemos tenido problemas al realizar esta “hibridación” con la fuente alternativa del Warehouse debido al diseño del Esquema Mediador. Solo hemos conseguido que en caso de no poder acceder a la fuente de datos en tiempo real, se muestre un mensaje de error y unos resultados parcialmente incompletos a partir del Warehouse.

8 Límites y retos del sistema

En esta sección se describen las principales limitaciones y retos encontrados durante el desarrollo del sistema integrador **Tren-ES**.

8.1 Limitaciones del sistema

Aunque el esquema mediador se ha diseñado para representar estaciones, municipios, rutas, paradas, viajes y distancias a nivel nacional, la demo final se ha limitado a un subconjunto de estaciones de Valladolid y Palencia.

Esta decisión se debe a que trabajar con todas las estaciones disponibles en RENFE resultaba inasumible tanto en tiempo como en espacio. El scraping de rutas entre pares de estaciones tardaba demasiado al crecer el número de combinaciones, y los cruces necesarios para generar datos como distancias, rutas y viajes producían un volumen de información demasiado grande para las máquinas virtuales utilizadas.

Por ello, los resultados obtenidos deben interpretarse como una demostración funcional del modelo de integración, no como una explotación completa de toda la red ferroviaria española.

Otra limitación aparece en la tercera consulta. La formulación original buscaba poblaciones con menos de cinco viajes programados, incluyendo aquellas con cero viajes. Para representar estos casos era necesario utilizar operaciones como **LEFT JOIN**, que no se ajustaban bien a la reformulación en forma conjuntiva. Por este motivo, la consulta se reformuló para considerar poblaciones con entre uno y cinco viajes programados.

8.2 Retos temporales

Uno de los principales retos ha sido la gestión del tiempo entre los distintos hitos de la práctica. El proyecto requería definir fuentes, construir el esquema mediador, plantear los mappings GAV y LAV, extraer datos reales, cargarlos y formular consultas sobre ellos.

El scraping aumentó esta dificultad, ya que no bastaba con definir las fuentes de forma teórica: era necesario obtener datos reales y transformarlos a un formato compatible con el sistema. Además, al tener que repetir el scraping para distintos pares de estaciones, el tiempo total de ejecución crecía rápidamente.

8.3 Retos en la extracción de datos

La extracción de datos desde ADIF y RENFE ha sido uno de los puntos más complejos del proyecto.

En ADIF, una petición HTTP básica no era suficiente, ya que la web bloqueaba el acceso automatizado. Fue necesario utilizar **playwright** para simular la navegación de un usuario real y poder acceder a las tablas de salidas y llegadas.

En RENFE, la información de horarios tampoco estaba disponible como una tabla descargable o una API sencilla. Fue necesario interactuar con un formulario, introducir origen, destino y fecha, y acceder después a páginas intermedias para obtener las paradas de cada recorrido.

Este proceso era especialmente costoso porque debía repetirse para cada par origen-destino. Al aumentar el número de estaciones, el número de combinaciones crecía muy rápido, haciendo inviable realizar el scraping completo.

8.4 Retos espaciales y de cruce de datos

Además del tiempo de ejecución, también hubo problemas relacionados con el espacio necesario para almacenar y cruzar los datos.

Relaciones como **Distancia** requieren considerar pares de estaciones. Esto implica que, al aumentar el número de estaciones, el número de tuplas posibles crece de forma muy rápida. Algo similar ocurre al cruzar estaciones, rutas, paradas y viajes procedentes de distintas fuentes.

Este motivo también llevó a la decisión de limitar la instancia de la demo. Así se evita que el tamaño de los datos derivados impida completar la carga y consulta del sistema.

8.5 Retos tecnológicos

Durante el desarrollo se han utilizado tecnologías nuevas para el grupo, como **playwright**, Airflow y GraphQL. Estas herramientas han requerido un proceso previo de aprendizaje para entender cómo aplicarlas dentro de un sistema integrador con fuentes heterogéneas.

También se han usado consultas SPARQL sobre WikiData para obtener información de municipios. Aunque esta fuente ofrece datos estructurados, fue necesario tener en cuenta detalles técnicos como el uso de cabeceras para evitar bloqueos.

8.6 Retos de integración

Las fuentes utilizadas no compartían el mismo modelo de datos ni los mismos identificadores. Por ello, fue necesario definir correspondencias entre los esquemas origen y el esquema mediador.

Por ejemplo, la tabla **Municipio** requiere combinar datos de WikiData y del INE. WikiData aporta población y coordenadas, mientras que el INE permite relacionar municipios con provincias y comunidades autónomas.

En el caso de RENFE, las paradas de una ruta aparecían como una lista no normalizada. Para adaptarlas al esquema mediador fue necesario introducir una vista auxiliar que permitiera descomponer esa lista y construir la relación **Parada**.

8.7 Otras limitaciones

Inicialmente se planteó añadir una interfaz gráfica para facilitar la consulta del integrador. Sin embargo, por falta de tiempo, la demo final no incluye una interfaz de usuario y se centra en mostrar el funcionamiento del sistema mediante los datos cargados y las consultas definidas.